Script Task

Why should I use scripts?

- With the extended scripting support it is easy to prototype complete processes including the business logic.
- Also the sharing of such prototypes is simple, because the whole code can be embedded inside the process.
- So only one file has to be exchanged and discussed.
- In combination with the BPMN model API it allows you to create mock process Script taskes with business logic directly inside your code.
- Another advantage is that now also non-Java programmers can use camunda BPM to execute their processes without writing Java code.
- We can reuse scripts by using the camunda:resource attribute. Which loads scripts from external resources.
- Either the scripts can be part of your deployment or reside in the classpath of your application.

Script support in BPMN

- The following table provides an overview of the BPMN elements which support the execution of scripts.

BPMN Element	Script Support
Script Task	Script inside the script task
Processes, Activities, Sequence Flows, Gateways and Events	Script as an execution listener
User Tasks	Script as task listener
Sequence Flows	Script as condition expression of a sequence flow
All Tasks, All Events, Transactions, Subprocesses and Connectors	Script inside an inputOutput parameter mapping

Script Task

- A Script Task is an automated activity. When a process execution arrives at the Script Task, the corresponding script is executed.

- A Script Task is defined by specifying the script and the Script Format.
- Supported script formats are Groovy, JavaScript, JRuby and Jython.
- All process variables can be accessible through the execution.
  - **execution.getVariable("variable name");**
- It's also possible to set process variables in a script.
  - **execution.setVariable("variable name","value");**

Script Task

- Information can be accessed through execution's default variable.
  - execution.eventName
  - execution.activityId
  - task.name
  - task.eventName

Variables Available During Script Execution

- During the execution of scripts, all process variables visible in the current scope are available.
- They can be accessed directly by the name of the variable (i.e., sum).
- There are also special variables:
  - execution, which is always available if the script is executed in an execution scope (e.g., in a script task) (DelegateExecution).
  - task, which is available if the script is executed in a task scope (e.g., a task listener) (DelegateTask).

Variables Available During Script Execution

Execution Variables:

- execution.activityInstanceId
- execution.businessKey
- execution.currentActivityId
- execution.currentActivityName
- execution.currentTransitionId
- execution.eventName
- execution.id
- execution.processBusinessKey
- execution.processDefinitionId
- execution.processInstanceId
- execution.tenantId

Execution methods:

- execution.setVariable(variableName, value);
- execution.getVariable(variableName)

Variables Available During Script Execution

Script:

- Script format: groovy

Script:

- out.println "Execution instance id : "+execution.activityInstanceId
- out.println "Business key : "+execution.businessKey
- out.println "Current activity id : "+execution.currentActivityId
- out.println "Current activity name : "+execution.currentActivityName
- out.println "Current transition id : "+execution.currentTransitionId
- out.println "Event name : "+execution.eventName
- out.println "Execution id : "+execution.id
- out.println "Process business key : "+execution.processBusinessKey
- out.println "Process definition id : "+execution.processDefinitionId
- out.println "Process instance id : "+execution.processInstanceId
- out.println "Tenant id : "+execution.tenantId

Variables Available During Script Execution

Task Variables:

- task.assignee
- task.caseExecutionId
- task.caseDefinitionId
- task.caseInstanceId
- task.caseDefinitionId
- task.createTime
- task.deleteReason
- task.description
- task.dueDate
- task.eventName
- task.executionId
- task.followUpDate
- task.id
- task.name